

1. Aim:

To write a C++ program to reverse a given string.

Algorithm:

- Step 1: Start the program.
- Step 2: Read a string from the user.
- Step 3: Find the length of the string.
- Step 4: Initialize two indices, one at the beginning and one at the end of the string.
- Step 5: Swap the characters at the beginning and end indices.
- Step 6: Move the beginning index forward and the end index backward.
- Step 7: Repeat Steps 5 and 6 until the indices meet or cross each other.
- Step 8: Display the reversed string.
- Step 9: Stop the program.

Program Code:

```
#include<iostream>
#include<string>
using namespace std;
int main(){
    string str;
    cout << "Enter a string: ";
    getline(cin, str);
    int n = str.length();
    for (int i = 0; i < n / 2; i++){
        char temp = str[i];
        str[i] = str[n - i - 1];
        str[n - i - 1] = temp;
    }
    cout << "Reversed String: " << str << endl;
    return 0;
}
```

Sample Output:

```
Z:\DS\Code\EX-3>
Z:\DS\Code\EX-3>g++ reverse.cpp -o reverse.exe

Z:\DS\Code\EX-3>reverse.exe
Enter a string: Hello
Reversed String: olleH
```

Result:

Thus, the C++ program to reverse a given string was executed successfully and the reversed string was displayed.

2. Aim:

To write a C++ program to reverse each word in a given sentence while maintaining the original word order.

Algorithm:

- Step 1: Start the program.
- Step 2: Read a string from the user.
- Step 3: Find the length of the string.
- Step 4: Initialize two indices, one at the beginning and one at the end of the string.
- Step 5: Swap the characters at the beginning and end indices.
- Step 6: Move the beginning index forward and the end index backward.
- Step 7: Repeat Steps 5 and 6 until the indices meet or cross each other.
- Step 8: Display the reversed string.
- Step 9: Stop the program.

Program code:

```
#include <iostream>
#include <string>
#include <algorithm>
using namespace std;
int main() {
    string str, word;
    cout << "Enter a sentence: ";
    getline(cin, str);
    for (int i = 0; i <= str.length(); i++) {
        if (i == str.length() || str[i] == ' ') {
            reverse(word.begin(), word.end());
            cout << word << " ";
            word = "";
        } else {
            word += str[i];
        }
    }
    return 0;
}
```

Sample Output:

```
Z:\DS\Code\EX-3>g++ revwordsen.cpp -o revwordsen.exe
```

```
Z:\DS\Code\EX-3>revwordsen.exe
Enter a sentence: I am Nithya
I ma ayhtiN
_
```

Result:

Thus, the C++ program to reverse each word in a sentence was executed successfully and the output was displayed.

3. Aim:

To write a C++ program to evaluate a postfix expression using a stack.

Algorithm:

- Step 1: Start the program.
- Step 2: Read the postfix expression from the user.
- Step 3: Create an empty stack.
- Step 4: Scan the expression from left to right.
- Step 5: If the current symbol is an operand, push it onto the stack.
- Step 6: If the current symbol is an operator, pop the top two operands from the stack.
- Step 7: Perform the required arithmetic operation.
- Step 8: Push the result back onto the stack.
- Step 9: Repeat Steps 5 to 8 until the entire expression is processed.
- Step 10: Display the element at the top of the stack as the final result.
- Step 11: Stop the program.

Program Code:

```
#include<iostream>
#include<stack>
#include<string>
using namespace std;
int main() {
    string exp;
    cout << "Enter Postfix Expression: ";
    cin >> exp;
    stack<int> s;
    for (char ch : exp) {
        if (isdigit(ch)) {
            s.push(ch - '0');
        } else {
            int val2 = s.top();
            s.pop();
            int val1 = s.top();
            s.pop();
            switch (ch) {
                case '+':
                    s.push(val1 + val2);
                    break;
                case '-':
                    s.push(val1 - val2);
                    break;
                case '*':
                    s.push(val1 * val2);
```

Ex No.:3

Date:

Poorna Sri.A

953625148038

```
        break;
    case '/':
        s.push(val1 / val2);
        break;
    }
}
}
cout << "Result = " << s.top() << endl;
return 0;
}
```

Sample Output:

```
Z:\DS\Code\EX-3>g++ postfix.cpp -o postfix.exe
```

```
Z:\DS\Code\EX-3>postfix.exe
Enter Postfix Expression: 23+
Result = 5
```

Result:

Thus, the C++ program to evaluate a postfix expression using a stack was executed successfully and the correct result was displayed.